

Parser

Tim Schmitz
25.10.2024



Inhalt

Dateien	3
Hauptprogramm	3
Initialisierung der Dateien	4
Erzeugen des Dictionary	4

Parser

Das Skript cbmParser.py ist ein Python-Programm, welches Daten aus dem Dateisystem ausliest und hieraus drei neue Dateien im Dateisystem erzeugt, um sie später in die CBM-Datenbank zu importieren.

Dateien

Die folgenden Dateien werden im Skript verwendet bzw. von dem Skript selbst erzeugt:

Dateiname	Dateityp	Bedeutung
ECO.txt	Input-Datei	Vorgegebene Liste von Eröffnungsschlüsseln aus Wikipedia
ecoFile.txt	Output-Datei	Indizierte Liste von Eröffnungsschlüsseln aus der Input-Datei
cbmFile.txt	Output-Datei	Indizierte Liste von CBM-Magazinnummern aus dem Verzeichnissystem
cbmEcoFile.txt	Output-Datei	Kombinierte Liste von CBM-Magazinnummern und Eröffnungsschlüsseln

Tabelle 1: Von dem Skript verwendete bzw. erzeugte Dateien

Als Speicherort im Dateisystem wird ein Verzeichnis innerhalb des Projektverzeichnisses gewählt, etwa C:\Users\username\Daten\Python\CBM\Files. Um die Dateien auszulesen bzw. zu erzeugen sind einige technische Vorbedingungen in dem Python-Skript notwendig, damit die gewünschten Funktionen programmgemäß arbeiten.

Die Input-Datei ECO.txt ist eine CSV-Datei (Comma Separated Value-Datei mit einem Semikolon als Trennzeichen) und hat folgendes Aussehen:

```
A24;1. c4 e5 2. Sc3 Sf6 3. g3 g6;  
A25;1. c4 e5 2. Sc3 Sc6;3. g3 g6 4. Lg2 Lg7 5. e3  
A26;1. c4 e5 2. Sc3 Sc6 3. g3 g6 4. Lg2 Lg7 5. d3;  
A27;1. c4 e5 2. Sc3 Sc6 3. Sf3;Dreispringer-Variante_ Holländische Struktur  
A28;1. c4 e5 2. Sc3 Sc6 3. Sf3 Sf6;Vierspringer-Variante
```

Dabei bezeichnet die erste Spalte den eindeutigen Schlüssel, nämlich den ECO-Code. Die zweite Spalte benennt die ersten Züge, die zu diesem Schlüssel gehören. Die dritte Spalte ist optional und dient der Erläuterung des Codes.

Hauptprogramm

Idealerweise steht das Hauptprogramm am Ende des Skripts, um keine zusätzlichen Ladezeiten im Python-Interpreter bereitstellen zu müssen. Das Hauptprogramm ist so kodiert, dass es einfach durchlaufen werden kann. Das Hauptprogramm besteht aus zwei Teilen:

1. Im ersten Teil werden die FileHandler für die eine Eingabedatei und die drei Ausgabedateien initialisiert
2. Im zweiten Teil werden das CBM-Dictionary und die Ausgabedateien erzeugt

Bei dem Dictionary handelt es sich um eine Datenstruktur in Python, die aus Schlüssel-Wert-Paaren besteht. Der erste Teil des Hauptprogramms besteht aus vier Aufrufen:

- Zunächst wird der Output-FileHandler für das CBM-File initialisiert, das ist diejenige Datei, die alle CBM-Ausgaben mit ihrer laufenden Nummer und einem fortlaufenden Index beinhaltet
- Anschließend erfolgt die Initialisierung des Input-FileHandlers für das ECO-Input-File. Diese Datei enthält alle ECO-Schlüssel mit Anfangszügen und einer Kurzbeschreibung aus Wikipedia
- Danach wird der Output-FileHandler für das ECO-Output-File erzeugt. Diese Datei entspricht der ECO-Input-Datei mitsamt Indizierung

- Zum Schluss wird der FileHandler für das CBM-ECO-File kreiert. Diese Datei beinhaltet alle Verweise auf Eröffnungsartikel und verknüpft sie mit der entsprechenden Information aus dem CBM- und dem ECO-File

Der erste Teil des Hauptprogramms besteht also aus vier Zeilen. Jede Zeile entspricht einer Datei als Rückgabewerte der Initialisierungs-Subfunktionen. Damit erfüllt jede der Subfunktionen eine genau abgegrenzte, enge Aufgabe.

```
#-----#
# Das Hauptprogramm ruft sequentiell die Unterprogramme #
# auf. Zunächst werden die FileHandler initialisiert. #
#-----#
cbmFile = initCbmFile()
ecoInputFile = initEcoInputFile()
ecoOutputFile = initEcoOutputFile()
cbmEcoFile = initCbmEcoFile()
```

Im zweiten Teil des Hauptprogramms werden die eigentlichen Funktionen des Skripts aufgerufen und abgearbeitet. Jede Funktion erzeugt eine Datei im Dateisystem.

1. Die Funktion createCbmEcoFile() erzeugt zunächst das CBM-Dictionary. Anschließend werden die Eröffnungsartikel identifiziert, indiziert und in das CBM-ECO-File geschrieben.
2. Die Funktion createEcoFile() erzeugt aus der vorgegebenen Liste der Eröffnungsschlüssel eine indizierte Liste derselben.
3. Die Funktion createCbmFile() erzeugt aus dem Verzeichnis der CBM-Magazine eine indizierte Liste aller CBM-Magazinnummern.

Der zweite Teil des Hauptprogramms besteht also aus drei Zeilen, die jeweils einem Funktionsaufruf entsprechen.

```
#-----#
# Anschließend werden das Dictionary und die #
# Ausgabedateien erzeugt #
#-----#
CbmDictionary = createCbmEcoFile()
createEcoFile(ecoInputFile, ecoOutputFile)
createCbmFile(CbmDictionary, cbmFile)
```

Initialisierung der Dateien

Zur Initialisierung von Input- oder Output-Dateien in Python werden FileHandler verwendet. Das ist im allgemeinen ein Begriff für die Handhabung von Dateien in Python, es gibt aber auch eine spezielle Verwendung des Begriffs. Dabei ist der FileHandler in Python ein Bestandteil des logging-Moduls und wird verwendet, um Log-Nachrichten in eine Datei zu schreiben.

Es gibt die folgenden Modalkennzeichen:

- Für das Lesen einer Datei: 'r' für read
- Für das Schreiben einer Datei: 'w' für write
- Für das Anhängen an eine Datei: 'a' für add

Im Grunde sind die FileHandler für Input- und Output-Dateien sehr ähnlich und unterscheiden sich nur durch ein Modalkennzeichen, das den Modus für diese Datei angibt. Der FileHandler für die Input-Datei ECO.txt wird in folgender Subroutine definiert, die diesen als Rückgabewert behandelt.

```
#-----#
# Die Funktion initEcoInputFile() initialisiert den #
# ECO-Input-Dateihandler des Programms #
#-----#
def initEcoInputFile():
    print("Initialisiere EcoInputFileHandler ...")
    ecoInputFile = open(FILE_LOCATION + 'ECO.txt', 'r')
    return ecoInputFile
```

Erzeugen des Dictionary

In der Routine createCbmEcoFile() erzeugt das sogenannte Dictionary. Ein Dictionary ist eine Datenstruktur, die aus einfachen Schlüssel-Werte-Paaren besteht.

Die folgende Abbildung beschreibt den Ablauf der Subfunktion und skizziert im wesentlichen den Ablauf der Verarbeitung vom Start links oben bis zum Ende rechts unten.

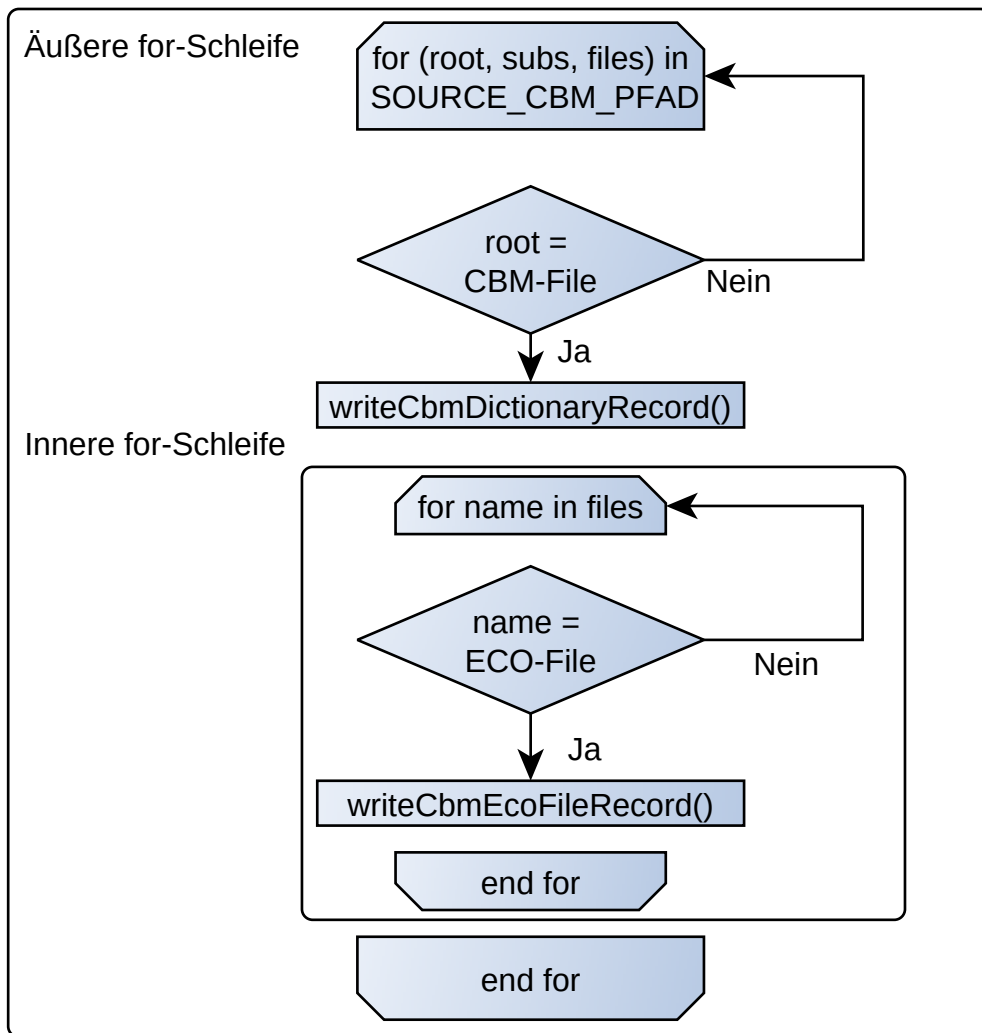


Abbildung 1: Ablauf der Subfunktion